

Emulazione come strumento virtuale VST del sintetizzatore analogico Minimoog

Davide Bianchi - 938447

Data di fine tirocinio: Dicembre 2021

1 Ente presso cui è stato svolto il lavoro di stage

L'ente dove è stato svolto il tirocinio è il Laboratorio di Informatica Musicale situato all'interno del dipartimento di informatica dell'Università degli Studi di Milano Statale.

2 Contesto iniziale

L'emulazione del Minimoog comporta la creazione del suono dei sintetizzatori analogici attraverso degli algoritmi implementati digitalmente. Le 2 componenti più importanti da emulare sono: il banco oscillatori e il filtro ladder. Per quanto riguarda il primo, in letteratura esistono diverse tecniche di sintesi: limitate in banda, quasi-limitate in banda e 'naive' (le quali inevitabilmente generano aliasing). L'approccio adottato per generare le forme d'onda fa parte della seconda categoria ed è chiamato Bandlimited Impulse Train (BLIT).

Per quanto riguarda il filtro, nel corso degli anni sono stati pubblicati articoli accademici (citati nell'elaborato) che propongono diversi modelli digitali. Alcuni di essi cercano di emulare il circuito analogico in dettaglio discretizzando le equazioni fisiche che lo descrivono (tecnica chiamata *white box modelling* o *modellazione fisica*), altri invece emulano solo il comportamento generale e le caratteristiche principali senza considerare la sua struttura interna (tecnica chiamata *black box modelling*). In questo progetto non è stata fatta una emulazione precisa del circuito ma è stato implementato un modello che rispecchiasse i comportamenti principali del filtro usando l'approccio *topology-preserving transform*. Commercialmente esistono 2 applicazioni degne di nota che emulano il comportamento del Minimoog: Minimoog model D di Moog Music Inc¹ sviluppata per dispositivi mobili IOS e Monark di Virtual Instrument².

3 Obiettivi del lavoro

L'obiettivo principale del lavoro era quello di sviluppare un plugin audio VST3 che emulasse il comportamento del Minimoog a livello sonoro e che replicasse le funzionalità più importanti cercando di mantenere un'interfaccia simile. Le informazioni sulla struttura del sintetizzatore sono state reperite dal manuale utente ufficiale [1]. Inoltre, una buona parte dell'elaborato riguarda lo studio e l'implementazione algoritmica della tecnica di sintesi Bandlimited Impulse Trains (BLIT) utilizzata per generare le forme d'onda del banco oscillatori del Minimoog. La maggior parte delle informazioni riguardante questo argomento sono state reperite dalla tesi di

¹<https://www.moogmusic.com/products/minimoog-model-d-synthesizer-app>

²<https://www.native-instruments.com/en/products/komplete/synths/monark/>

dottorato di Timothy Stilson [2]. Dopo alcuni mesi è stato scelto dal candidato di implementare anche l'algoritmo del filtro ladder del Minimoog dal momento che non esistevano già classi che emulassero il filtro completamente. La totalità delle informazioni legate allo sviluppo di filtri 'musicali' sono state reperite dal libro: 'The art of VA filter design' [3].

4 Descrizione del lavoro svolto

Il lavoro svolto si può riassumere nei seguenti passaggi:

- Analisi dello schema circuitale del Minimoog e delle principali caratteristiche e controlli da implementare.
- Sviluppo di un prototipo software iniziale che rispecchiasse la struttura del sintetizzatore analogico attraverso il paradigma della programmazione ad oggetti. In questa fase è stato deciso quante e quali classi creare, la loro interfaccia pubblica e la loro interazione.
- Stesura del codice che si occupa dell'elaborazione audio. Ovviamente non è stato implementato da zero tutto il sintetizzatore ma sono state utilizzate alcune classi della libreria di JUCE come *Synthesiser* o *ADSR*. Inizialmente l'algoritmo degli oscillatori era naïve e generava una grande quantità di aliasing sempre più percepibile all'aumento della frequenza. Anche il filtro non è stato implementato sin da subito ma è stata usata temporaneamente la classe *LadderFilter* di JUCE.
- Creazione della classe *Blit* la quale si occupa della generazione delle forme d'onda presenti nel banco oscillatori del Minimoog. L'algoritmo di sintesi implementato dalla classe è chiamato Bandlimited Impulse Train e permette la generazione di onde limitate in banda.
- Creazione della classe *MoogFilter* nella quale è implementato l'algoritmo del filtro ladder del Minimoog. Esso è stato implementato usando la l'approccio *topology-preserving transform* il quale permette di costruire il modello digitale del filtro non solo con la stessa funzione di trasferimento rispetto ma anche con la stessa struttura del circuito analogico originale.
- Testing dell'algoritmo di generazione delle forme d'onda e del filtro. Per quanto riguarda il primo è stata verificata l'accordatura degli oscillatori e la ridotta presenza di aliasing comparando la tecnica BLIT rispetto la tecnica di sintesi naïve. Nel secondo caso, invece, è stata verificata la risposta in frequenza del filtro a diverse frequenze di taglio.

5 Tecnologie coinvolte

Per lo sviluppo di questo progetto sono state utilizzate le seguenti tecnologie:

- C++³ è un linguaggio di programmazione general purpose sviluppato in origine da Bjarne Stroustrup nei Bell Labs nel 1983 come evoluzione del linguaggio C inserendo la programmazione orientata agli oggetti. Esso è un linguaggio compilato, flessibile e soprattutto efficiente, caratteristica fondamentale nell'elaborazione digitale dei segnali e quindi nella programmazione audio.
- JUCE⁴ è un framework open source e multi piattaforma utilizzato per lo sviluppo di plugin audio, di GUI, ma anche di applicazioni desktop e mobile in C++.

³<https://www.cplusplus.com/>

⁴<https://juce.com/>

- Projucer è un ambiente di sviluppo usato per creare e gestire i progetti JUCE. Una volta specificati i file e le impostazioni di un progetto, esso si occupa di generare un insieme di file per permettere al codice di essere compilato nativamente su ogni piattaforma. Al momento JUCE supporta i seguenti ambienti di sviluppo: Visual Studio, Xcode e CodeBlocks.
- Visual Studio⁵ è un ambiente di sviluppo integrato creato da Microsoft. Esso è utilizzato per lo sviluppo di programmi desktop, mobile e applicativi web. Visual Studio ha un editor che supporta Intellisense (auto completamento) e il refactoring del codice. Esso dispone di un debugger, di un profiler, di un ambiente per progettare interfacce grafiche, pagine web, diagrammi di classi e di database. Permette l'uso di plug-ins che espandono le sue funzionalità e l'uso di sistemi come Git o Azure. Esso supporta 36 tipi di linguaggi di programmazione diversi.
- La DAW (Digital Audio Workstation) è un software usato per la registrazione, l'editing e la produzione di file audio. Essa è stata usata per caricare ed eseguire il plugin e testarlo tramite l'utilizzo di strumenti come l'oscilloscopio e l'analizzatore di spettro.
- MATLAB⁶ è un linguaggio di programmazione proprietario multiparadigma e un ambiente di calcolo numerico sviluppato da MathWorks. Esso consente manipolazioni di matrici, tracciatura di funzioni e dati, implementazione di algoritmi, creazione di interfacce utente e interfacciamento con programmi scritti in altri linguaggi. MATLAB è stato utilizzato per analizzare lo spettro degli oscillatori e la risposta in frequenza del filtro.

6 Competenze e risultati raggiunti

Durante questo lavoro è stato possibile seguire il processo di sviluppo di un plugin audio partendo dalla sua struttura ad alto livello, sviluppando poi gli algoritmi DSP e infine testando le varie funzionalità. Gli obiettivi posti all'inizio del progetto sono stati tutti raggiunti. Ogni funzionalità presente nel pannello del Minimoog è stata testata con la DAW Ableton Live e l'algoritmo degli oscillatori e del filtro anche con MATLAB. Le onde generate dagli oscillatori rispecchiano correttamente quelle della versione analogica e l'aliasing risulta impercettibile anche a frequenze elevate. Il filtro ha una risposta in frequenza che rispecchia quella del ladder del Minimoog, inoltre, è possibile modularlo a frequenze audio. La maggior parte dei problemi riscontrati riguardano la stesura dell'algoritmo del BLIT e del filtro dal momento che sono stati creati da zero ed era necessario capire bene la teoria in modo da effettuare correttamente la stesura del codice e velocizzare la fase di debugging. Ci ha retroazione che collega l'uscita del sintetizzatore al mixer e la possibilità dell'oscillatore 3 di automodularsi in frequenza. Migliorare triangolare e il prewarping del filtro.

Riferimenti bibliografici

- [1] MoogMusic. *Minimoog model D user's manual*
- [2] Timothy Scott Stilson. *Efficiently-variable non oversampled algorithms in virtual music synthesis - A root locus perspective*. PhD thesis, Giugno 2006.
- [3] Vadim Zavalishin. *The art of VA filter design*. 2.1.0 edition, Ottobre 2018.

⁵<https://visualstudio.microsoft.com/it/>

⁶<https://it.mathworks.com/products/matlab.html>